

Strukturalni dizajn paterni

Adapter

Osnovna namjena ovog paterna je da omogući širu upotrebu već postojećih klasa. Ovo je korisno u situacijama kada je potreban drugačiji interfejs već postojeće klase, koju ne želimo mijenjati. Ovaj pattern kreira novu adapter klasu koja djeluje kao posrednik između originalne klase i traženog interfejsa.

U našem sistemu ovaj patern bi se mogao primijeniti ukoliko bi se uveo vanjski servis za slanje e-mail obavještenja koji koristi drugačiji interfejs od postojeće klase Obavjestenje. U tom slučaju bi se kreirala adapter klasa koja prevodi pozive iz sistema u format koji vanjski servis razumije, bez mijenjanja postojeće klase Obavjestenje.

Facade

Facade patern pruža pojednostavljen interfejs prema složenom podsistemu. Umjesto da klijentski kod direktno komunicira s više klasa i poznaje njihov redoslijed pozivanja, Facade nudi jedan objekat koji sakriva internu kompleksnost i koordinira sve potrebne operacije unutar podsistema.

U sistemu biblioteke operacije kao što su posudba i vraćanje knjige zahtijevaju koordinaciju više klasa istovremeno. Operacija posudbe knjige uključuje: provjeru dostupnosti primjerka (klasa Primjerak), kreiranje zapisa o posudbi (klasa Posudba), promjenu statusa primjerka te slanje obavještenja korisniku (klasa Obavjestenje). Bez Facade paterna, svaki kontroler bi morao poznavati i koordinirati sve ove korake. Implementacijom klase BibliotekeFacade s metodama posudiKnjigu(), vratiKnjigu() i produziPosudbu(), kontroler poziva samo jednu metodu, dok interna logika ostaje skrivena iza fasade.

Decorator

Decorator patern omogućava dinamičko dodavanje novih funkcionalnosti objektima bez izmjene njihovog izvornog koda. Umjesto kreiranja velikog broja podklasa, ovaj patern fleksibilno 'umotava' objekte dodatnim ponašanjima putem posebnih dekoratorskih klasa, zadržavajući pritom princip otvorenosti za proširenje.

Ovaj patern bi se mogao primijeniti na klasu Knjiga kako bi se dinamički proširio prikaz informacija o knjizi. Na primjer, jedan dekorator bi mogao dodati prikaz prosječne ocjene i recenzija uz osnovne podatke o knjizi, dok bi

drugi dekorator dodao informacije o dostupnosti primjeraka i trenutnom statusu. Na ovaj način bi se osnovni prikaz knjige mogao fleksibilno proširivati bez mijenjanja same klase Knjiga.

Bridge

Bridge patern služi za odvajanje apstrakcije od njene implementacije, što omogućava da se obje strane mijenjaju nezavisno jedna od druge. Naročito je pogodan kada klasa može imati više različitih apstrakcija i više različitih implementacija.

Ovaj patern bi se u našem sistemu mogao primijeniti za odvajanje prikaza knjiga od načina dohvata podataka. Registrovani korisnik bi vidio detaljan prikaz knjige s informacijama o dostupnosti, ocjenama i mogućnošću rezervacije, dok bi gost vidio samo osnovne informacije. Koristeći Bridge patern, apstrakcija prikaza bi se odvajala od logike dohvata podataka, što bi omogućilo nezavisno proširivanje oba aspekta bez međusobnih ovisnosti.

Proxy

Proxy patern uvodi zamjenski objekat koji kontrolira pristup stvarnom objektu. Proxy implementira isti interfejs kao i stvarni objekat tako da klijent ne mora znati da li komunicira direktno sa stvarnim objektom ili s proxyjem. Na ovaj način Proxy može vršiti provjere pristupa prije nego proslijedi poziv stvarnom objektu.

Klasa Korisnik u našem sistemu sadrži osjetljive metode poput `obrisiKorisnika()`, `dodajKorisnika()` i `editujKorisnika()`. Sistem ima tri korisničke uloge — bibliotekar, administrator i korisnik — pri čemu svaka uloga ima različita prava pristupa. Uvođenjem interfejsa `IKorisnik` i klase `KorisnikProxy`, kontrola pristupa je centralizovana na jednom mjestu. `KorisnikProxy` provjerava ulogu korisnika i tek tada proslijedi poziv stvarnoj klasi `Korisnik` — npr. samo administrator smije pozvati `obrisiKorisnika()`.

Composite

Composite patern omogućava kreiranje hijerarhijskih struktura u obliku stabla, gdje se pojedinačni objekti i kompozicije tih objekata tretiraju na isti način. Koristi se kada je potrebno uniformno upravljati i pojedinačnim elementima i grupama elemenata.

U okviru našeg sistema ovaj patern bi se mogao primijeniti za organizaciju knjiga u hijerarhiju kategorija i podkategorija. Na primjer, žanr 'Fantastika' mogao bi sadržavati podkategorije 'Naučna fantastika' i 'Visoka fantastika', a svaka od njih konkretne knjige. I kategorije i pojedinačne knjige bi implementirale isti interfejs, što bi omogućilo uniformno pretraživanje i upravljanje cijelom hijerarhijom bez razlikovanja da li je u pitanju pojedinačna knjiga ili cijela kategorija.